Money Tracking Model (A1)

Proposal for Owner and PM review | written 22 August 2026, reissued for Khadijah 16 October 2026 | nothing in this area is built until this is approved

Peter's instruction, Decision Log item 7 / A1: before refunds, milestone payments and payouts are built, agree ONE financial ledger model that tracks every money event as a discrete, separately auditable record, and never represent the picture by editing a single booking total field. This is that proposal.

1. The one idea

> **A booking total is never stored and edited. It is always the sum of a list of money events.** Every charge, fee, commission, upgrade, refund and payout is one append-only row. To correct something you add a reversing row; you never edit or delete history. The current total is a question we answer by adding up the list, so it can never silently drift from what actually happened.

This is the opposite of the trap Peter named: one booking total field that gets overwritten, where last month's number is gone and nothing reconciles.

2. What exists today, and why it is not enough

TODAY	WHY IT CANNOT CARRY THIS
payments	A gateway charge, but tied to a subscription, so it cannot cleanly record a booking fee, an upgrade or a refund.
payouts	A professional withdrawal, flat amount and status, with no link to which bookings funded it.
Commission	Calculates the 5 / 3 / 1.5 percent share, but nothing records it.

There is no place that ties request fee to booking to commission to payout as auditable line items. That is the gap.

3. Proposed tables

3a. ledger_entries, the spine (append-only)

One row per money event. Never updated after it settles; corrections are new rows.

COLUMN	PURPOSE
event_id (nullable)	the request this concerns
booking_id (nullable)	the booking this concerns
party_user_id	whose money this entry is about: Client, Professional, or GigResource
entry_type	one of the nine kinds below
amount	decimal(12,2), always positive
sign	money in or money out, from GigResource's books
currency	
state	pending, then secured, then released or refunded or failed
subject_type / subject_id	link to a Booking, Upgrade, Milestone, Payment or Payout
payment_id (nullable)	the gateway charge that backs it
reverses_id (nullable)	set when this row corrects an earlier one
metadata	provider references, tax jurisdiction, notes
occurred_at	

The nine records Peter asked for, one entry type each:

- **booking_charge** the original booking amount
- **platform_fee** the Client platform fee, including the posting fee
- **commission** GigResource's share, at the professional's tier rate

- **upgrade** a mid-event upgrade or change order, each its own row, never merged
- **provider_fee** the Stripe or PayPal fee
- **tax** applicable tax
- **refund / adjustment** a reversal or correction
- **milestone_funding / milestone_release** money parked for, then released to, a stage
- **payout** money sent to the professional

3b. booking_upgrades, each upgrade its own record (the B3 rule)

Peter confirmed in B3 that a mid-event upgrade is an amendment to the existing booking, shown to the Client as one running total but recorded internally as separate line items, never collapsed. Payment must be secured before the upgrade is confirmed.

COLUMN	PURPOSE
booking_id, requested_by	
description, amount	what and how much
commission_rate	the pro's ordinary tier rate. No special upgrade fee, per B3.
status	requested, approved, payment_secured, confirmed, or rejected
secured_payment_id	the charge that gated confirmation

The Client-facing booking total is derived: the original charge plus the confirmed upgrade rows. There is no modified total field.

3c. milestones, only when milestone payments are built

Label, amount, and a state of planned, then funded, then released. Each funding and release is also a ledger row.

3d. payments, one small change

Make the subscription link optional and add a purpose: subscription, request fee, booking, upgrade or refund. A payment can then back a ledger entry that is not a subscription. Existing subscription payments are unaffected.

4. The flow, when everything goes right

STEP	WHAT IS RECORDED
Client posts a request	platform_fee entry, pending then secured
Client finalises a bid	booking_charge secured with the deposit held, and commission accrued but not yet taken
Mid-event upgrade	an upgrade row requested then approved by the Client; payment secured, then the upgrade confirmed and the total recomputed
Work marked complete	milestone or final release, provider_fee and tax recorded, then a payout entry of booking plus upgrades less commission and fees
Something goes wrong	a refund entry that reverses the specific entries, by reference

Money owed, held, earned and paid are all read by filtering the ledger. None of them is a stored field anyone overwrites.

5. How this satisfies Peter's rule

- **Discrete and auditable** every one of the nine is its own row, with its own state and its own provider reference.
- **Nothing merged** upgrades never fold into the booking total; the total is a sum, computed on read.
- **Nothing overwritten** corrections are reversing rows, so the history reconciles and an auditor can trace every dollar.

6. Open questions the Owner still needs to settle

#	QUESTION
1	**Fee timing.** The Decision Log confirms the posting fee is charged at publish, not at finalisation, but A1 also lists it as a ledger item to design here first, so the two overlap. Recommendation: build it as a platform_fee entry created at publish, once this model is approved. It is deliberately not moved yet, to avoid building money records ahead of this review.

2	**Who holds the money.** Does GigResource hold funds in escrow, or does the provider hold them and we only record? This decides whether "held" is real custody or a bookkeeping state.
3	**Refund policy.** Windows, and who can trigger one. Dispute deadlines are reserved for attorney review; no screen states one and this model should not hard-code one either.
4	**Payout timing.** On completion, on a schedule, or on request, and any minimum threshold.
5	**Taxes.** Do we compute and collect, or record only, and which jurisdictions in version one.

7. Build order once approved

1. ledger_entries and the read helpers for money owed, held, earned and paid.
2. Backfill the existing request fee and any finalisation charges as entries.
3. booking_upgrades and the secure-before-confirm gate, which unblocks the B3 upgrade billing.
4. Move the posting fee to publish as a platform_fee entry, per B1.
5. Refunds as reversing entries, then milestones, then payout reconciliation.

> **Nothing above is started until the Owner and PM approve this model.** That is why every pricing decision taken in October is still waiting: the prices are settled, but there is nowhere to record them.

Written 22 August 2026 as docs/money-ledger-proposal.md in the repository. Reissued unchanged in substance for Khadijah on 16 October 2026, with the pricing context added.